

Blockchain B-tree Indexer

Sistema de Indexação Eficiente para Dados de Blockchain

João Lucas Pinto

Luiza de Araújo Nunes Gomes

Matheus Bezerra

Rafael da Silva Oliveira

Introdução

Este projeto apresenta uma implementação de um sistema de indexação baseado em B-trees para dados de blockchain, desenvolvido com interface interativa em Streamlit. Inspirado no trabalho pioneiro de Du et al. (2021) no artigo "EtherH: A Hybrid Index to Support Blockchain Data Query", o sistema demonstra como estruturas de dados clássicas de bancos de dados podem ser aplicadas para resolver um dos maiores desafios das tecnologias de registro distribuído: a consulta eficiente de dados.

Contexto e Problema

Blockchains, como o Ethereum, armazenam dados em estruturas sequenciais (ex.: LevelDB), que oferecem suporte limitado a consultas complexas. Conforme destacado no artigo EtherH, a falta de índices eficientes restringe aplicações em áreas como finanças e supply chain, onde consultas rápidas por valor único (Single-V query) ou intervalos (Range query) são essenciais.

Introdução

Solução Proposta

A proposta foca no uso de uma B-tree pura para otimizar consultas em blockchain. A ideia central é melhorar a inserção eficiente e garantir consultas rápidas, indo além da complexidade $O(n)$ para atingir $O(\log n)$. O sistema também oferece personalização, permitindo a indexação de atributos específicos da blockchain, como "GasUsed" e "Timestamp", para atender às necessidades do usuário.

Diferenciais

- Interface Interativa: Diferente do EtherH (implementado em Geth v1.9), este projeto utiliza Streamlit para visualização em tempo real da árvore e métricas de desempenho.
- Simplicidade vs. Híbrido: Enquanto o EtherH usa uma abordagem híbrida (B-tree + Skip-list), esta implementação prioriza a B-tree para fins didáticos, mantendo a eficiência em operações de disco (redução de I/O, conforme Section 3.1 do artigo).

Objetivos

- Demonstrar a aplicabilidade de B-trees em blockchains, complementando os resultados experimentais do EtherH (que mostrou consumo de armazenamento de apenas 700 MB para 4M blocos).
- Oferecer uma ferramenta educacional para estudo de estruturas de dados em contextos descentralizados

A implementação simula um blockchain com todas as suas características fundamentais, combinado com múltiplos índices B-tree, oferecendo uma interface web intuitiva para demonstração das funcionalidades.

Blockchain: A Arquitetura de Registros Imutáveis

Blockchains são estruturas de dados distribuídas que armazenam uma lista crescente de blocos. Cada bloco, que contém um hash criptográfico do bloco anterior, um timestamp e dados de transação, é encadeado e protegido por criptografia. Essa arquitetura torna os blockchains resistentes a modificações, garantindo um registro de transações imutável e transparente.

Cada bloco contém:

- Hash criptográfico do bloco anterior;
- Timestamp (marca temporal);
- Dados das transações.

Limitação para consultas:

A estrutura linear apresenta complexidade $O(n)$ para buscas, tornando-se um gargalo em blockchains de grande escala.

B-trees: A Solução Clássica para Indexação de Dados

B-trees são estruturas de dados em árvore, auto-balanceadas, que mantêm os dados ordenados e permitem operações em tempo logarítmico.

Vantagens para indexação de blockchain:

- Redução da complexidade de $O(n)$ para $O(\log n)$
- Suporte nativo a consultas por intervalo (range queries)
- Estrutura otimizada para sistemas que leem grandes blocos de dados

Para um blockchain com um milhão de transações, a melhoria pode significar uma redução de até 50.000 vezes no número de operações necessárias.

A Arquitetura de Indexação Múltipla

O sistema implementado utiliza uma estratégia de indexação múltipla, mantendo diferentes índices B-tree para diferentes tipos de consulta.

Índice por ID de Transação:

Para buscas exatas e rápidas de uma transação específica.

Índice por Timestamp:

Para consultas temporais e por intervalo de datas.

Índice por Remetente:

Para análises de fluxo de fundos e histórico de transações.

Índice por Destinatário:

Para rastrear os fundos recebidos por um usuário.

Implementação do Blockchain

O sistema é composto por três classes principais:

Transaction:

Representa uma transação individual

Contém sender, receiver, amount e timestamp

Gera transaction_id único usando hash SHA-256

Block:

Unidade fundamental do blockchain

Contém index, timestamp, transactions, previous_hash

Implementa algoritmo de Proof of Work simplificado

Blockchain:

Gerencia a cadeia completa de blocos

Adiciona transações pendentes e minera novos blocos

Valida a integridade da cadeia

Implementação do Sistema de Indexação B-tree

O módulo **btree.py** contém duas classes principais:

BTreeNode:

- Representa um nó individual na estrutura da B-tree

- Mantém listas de keys, values e children

- Implementa inserção ordenada e operação de split

BTree:

- Implementa a lógica completa da B-tree

- Gerencia a raiz e coordena operações

- Suporta chaves duplicadas e consultas por intervalo

Integração Blockchain-Indexer

A classe **BlockchainIndexer**

é o elo crucial que conecta o blockchain com o sistema de indexação B-tree:

- **Inicialização:**
Cria uma instância de Blockchain e inicializa quatro B-trees separadas, cada uma com uma finalidade específica.
- **Indexação Automática:**A função `_index_block(block)` é chamada automaticamente sempre que um novo bloco é minerado e adicionado à cadeia.
- **Métodos de Consulta:**
Expõe métodos de alto nível para a aplicação Streamlit realizar consultas, como:
`get_transaction_by_id`
`get_transactions_by_sender`

Esta integração garante que os índices estejam sempre atualizados e prontos para consultas eficientes.

Interface Streamlit

A aplicação **app.py**

É a interface do usuário construída com Streamlit, que permite a interação visual com o blockchain e seus índices B-tree:

As seções de consulta (por ID, remetente, destinatário, período) mostram a eficiência das B-trees na recuperação rápida de dados.

O Streamlit atua como meio pelo qual o usuário pode observar e interagir com a atuação conjunta do blockchain e seus índices B-tree.

Componentes Interativos:

Visualização de Dados:

Demonstração de Consultas:

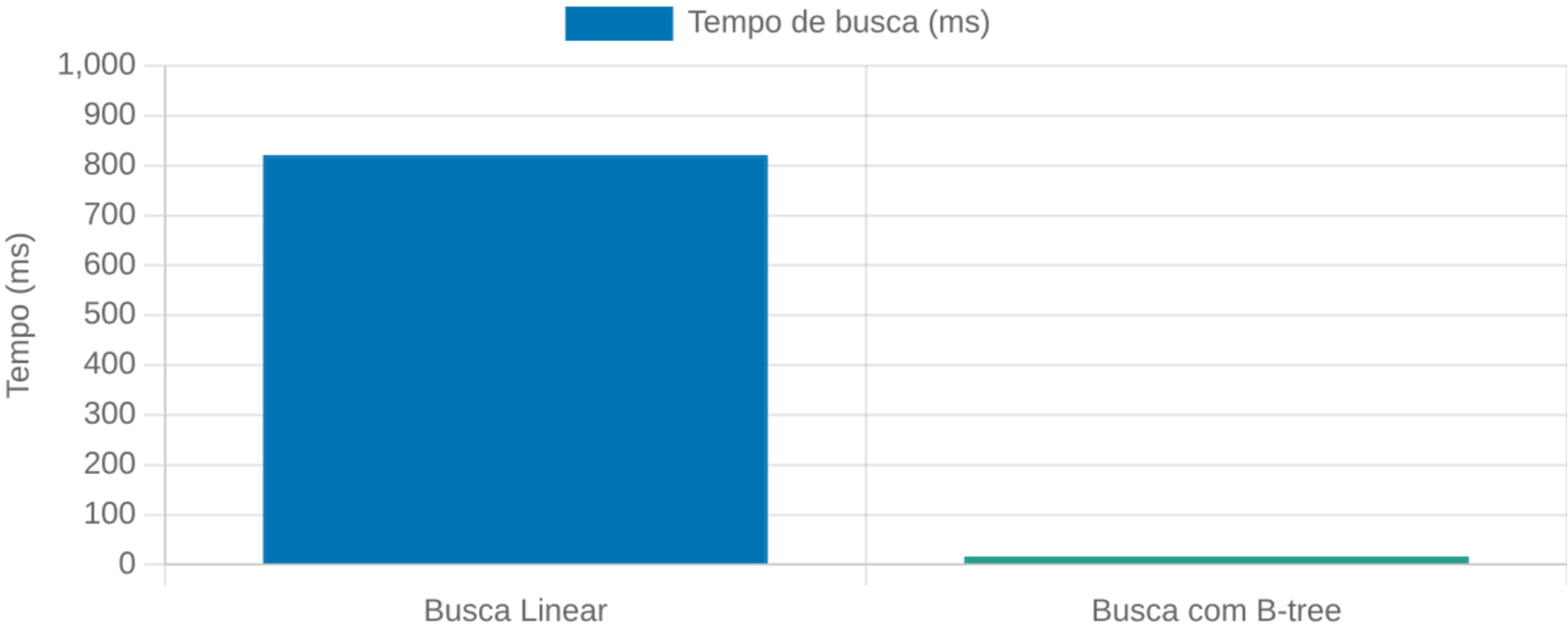
Análise de Performance

A implementação de índices B-tree oferece melhorias dramáticas de performance:

Método	Complexidade	Descrição
Sem Índice	$O(n)$	Varredura linear em toda a blockchain
Com B-tree	$O(\log n) + O(k)$	k = número de transações do remetente

Otimizações de Memória:

- Referências aos dados originais em vez de cópias completas
- Cache para nós frequentemente acessados
- Indexação seletiva apenas dos campos relevantes



Comparação de tempo de busca (ms) com 1 milhão de transações

Casos de Uso e Aplicações Práticas

Auditoria e Compliance Financeiro

Rastreamento rápido de fluxo de fundos e análise de atividades durante períodos específicos. Essencial para investigações financeiras e verificação de origem de transações.

Geração de Relatórios Regulatórios

Organizações financeiras podem gerar rapidamente relatórios sobre atividades específicas, cumprindo requisitos regulatórios sem necessidade de varreduras completas da cadeia.

Análise de Dados e Business Intelligence

Extração de insights valiosos sobre padrões de uso e comportamento da rede. Identificação de tendências, picos de atividade e análise de grafos de transação.

Aplicações de Pagamento e Carteiras Digitais

Recuperação rápida do histórico de transações de um usuário, busca avançada em carteiras e reconciliação automática de saldos e histórico de transações.

O sistema de indexação de blockchain com B-trees abre um leque de possibilidades para aplicações práticas, resolvendo problemas de performance que são inerentes à natureza sequencial dos blockchains.

Conclusão

Principais conclusões do projeto:

- ✓ A integração de B-trees com blockchain resolve o problema crítico de consultas eficientes, reduzindo a complexidade de $O(n)$ para $O(\log n)$.
- ✓ A estratégia de indexação múltipla permite otimizar diferentes tipos de consulta sem comprometer a performance geral.
- ✓ O overhead de manutenção dos índices é mínimo comparado aos ganhos de performance nas consultas subsequentes.
- ✓ A interface Streamlit demonstra de forma prática como a indexação transforma a experiência do usuário em aplicações blockchain.

Referências

ALEXANDRE, R. **Bitcoin Core Documentation**. 2023. Disponível em: <https://bitcoincore.org/en/doc/>. Acesso em: 12 jun. 2025.

ANDERSON, R. **Security Engineering: A Guide to Building Dependable Distributed Systems**. 2. ed. Indianapolis: Wiley, 2008.

BAKA, B. **Python Data Structures and Algorithms**. 1. ed. Birmingham: Packt Publishing, 2023. Acesso em: 05 jun. 2025.

BASHIR, I. **Mastering Blockchain: A Deep Dive into Distributed Ledgers**. 3. ed. Birmingham: Packt Publishing, 2022. Acesso em: 18 jun. 2025.

COMER, D. **B-Trees and Relational Database Systems**. Berkeley: University of California Press, 1981. Acesso em: 03 jun. 2025.

CORMEN, T. H. et al. **Introduction to Algorithms**. 4. ed. Cambridge: MIT Press, 2022. Acesso em: 15 jun. 2025.

DU, Pengting et al. **EtherH: A Hybrid Index to Support Blockchain Data Query**. In: ACM TURING CELEBRATION CONFERENCE – CHINA (ACM TURC '21), 30 jul.–1 ago. 2021, Hefei, China. Anais [...]. New York: ACM, 2021. p. 1-5. Disponível em: <https://doi.org/10.1145/3472634.3472653>. Acesso em: 10 jun. 2025.

ELASTIC. **Elasticsearch: The Definitive Guide**. 2023. Disponível em: <https://www.elastic.co/guide/>. Acesso em: 08 jun. 2025.

ETHEREUM. **Ethereum Yellow Paper**. 2023. Disponível em: <https://ethereum.github.io/yellowpaper/paper.pdf>. Acesso em: 20 jun. 2025.

Referências

GARCIA-MOLINA, H. **Database System Implementation**. 2. ed. Upper Saddle River: Prentice Hall, 2015. Acesso em: 10 jun. 2025.

GRAEFE, G. **Modern B-Tree Techniques**. Hanover: Now Publishers, 2011. Acesso em: 01 jun. 2025.

GRINBERG, M. **Flask Web Development: Developing Web Applications with Python**. 2. ed. Sebastopol: O'Reilly, 2018. Acesso em: 14 jun. 2025.

HYPERLEDGER. **Hyperledger Fabric Documentation**. 2023. Disponível em: <https://hyperledger-fabric.readthedocs.io/>. Acesso em: 07 jun. 2025.

IBM. **Blockchain Data Analytics For Dummies**. São Paulo: Alta Books, 2021. Acesso em: 19 jun. 2025.

KNUTH, D. E. **The Art of Computer Programming: Volume 3 - Sorting and Searching**. 2. ed. Boston: Addison-Wesley, 1998. Acesso em: 06 jun. 2025.

KLEPPMANN, M. **Designing Data-Intensive Applications**. Sebastopol: O'Reilly, 2017. Acesso em: 09 jun. 2025.

MONGODB. **MongoDB Indexing Strategies**. 2023. Disponível em: <https://www.mongodb.com/docs/manual/indexes/>. Acesso em: 16 jun. 2025.

Referências

OWASP. **OWASP Top 10**. 2023. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 04 jun. 2025.

POSTGRESQL. **PostgreSQL Documentation**. 2023. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 11 jun. 2025.

SCHWARTZ, B. et al. **High Performance MySQL**. 4. ed. Sebastopol: O'Reilly, 2022. Acesso em: 13 jun. 2025.

SOLANA. **Solana Whitepaper**. 2023. Disponível em: <https://solana.com/solana-whitepaper.pdf>. Acesso em: 02 jun. 2025.

SQLITE. **SQLite Documentation**. 2023. Disponível em: <https://www.sqlite.org/fileformat2.html>. Acesso em: 17 jun. 2025.

STALLINGS, W. **Cryptography and Network Security: Principles and Practice**. 8. ed. Boston: Pearson, 2023. Acesso em: 20 jun. 2025.

STREAMLIT. **Streamlit Documentation**. 2023. Disponível em: <https://docs.streamlit.io/>. Acesso em: 01 jun. 2025.

THE GRAPH. **The Graph Protocol**. 2023. Disponível em: <https://thegraph.com/>. Acesso em: 10 jun. 2025.

WOOD, G. **Ethereum: A Secure Decentralised Generalised Transaction Ledger**. Berlin: Ethereum Foundation, 2023. Acesso em: 05 jun. 2025.

Classe Transaction

```
1 class Transaction:
2     def __init__(self, sender: str, receiver: str, amount: float, transaction_id: str = None):
3         self.sender = sender                                # Remetente
4         self.receiver = receiver                            # Receptor
5         self.amount = amount                                # Valor da transação
6         self.timestamp = time.time()                        # Momento do envio
7         self.transaction_id = transaction_id or self._generate_transaction_id()
8         # Transaction ID -> (De até 16 bits) Se a ID for None, gera uma ID

1     def _generate_transaction_id(self) -> str:                # "Gera um ID único para a transação.
2         data = f"{self.sender}{self.receiver}{self.amount}{self.timestamp}"
3         """
4         data.encode()                # A string de dados da transação é codificada em bytes.
5         sha256(data.encode())        # A função de hash SHA-256 é aplicada aos bytes codificados.
6         hashlib.sha256(data.encode()).hexdigest()[:16]
7         # O hash é convertido para uma representação hexadecimal e truncado para os primeiros 16 caracteres.
8         """
9         return hashlib.sha256(data.encode()).hexdigest()[:16]
```

Classe Block

```
1 class Block:                                # Representa um bloco no blockchain.
2     def __init__(self, index: int, transactions: List[Transaction], previous_hash: str):
3         self.index = index                    # índice do bloco, posição do bloco na cadeia do blockchain
4         self.timestamp = time.time()          # Momento da CRIAÇÃO do bloco
5         self.transactions = transactions      # Lista de transações incluídas no bloco
6         self.previous_hash = previous_hash    # Bloco anterior à ele na cadeia
7         self.nonce = 0                       # "Number ONCE" = 0, vai ser utilizado pra encontrar o hash do bloco válido
8         self.hash = self._calculate_hash()    # Introduz o hash do bloco inicial.
```

```
1 def _calculate_hash(self) -> str:            # Calcula o hash do bloco.
2     block_string = json.dumps({
3         'index': self.index,
4         'timestamp': self.timestamp,
5         'transactions': [tx.to_dict() for tx in self.transactions],
6         'previous_hash': self.previous_hash,
7         'nonce': self.nonce},
8     sort_keys=True)
9     """
10    block_string.encode()                      # Codifica o JSON em bytes
11    sha256(block_string.encode())              # Aplica a função de hash SHA-256
12    hashlib.sha256(block_string.encode()).hexdigest() # Retorna em hexadecimal
13    """
14    return hashlib.sha256(block_string.encode()).hexdigest()
```

Classe Block

```
1 def mine_block(self, difficulty: int = 4):
2     """
3     Simula o processo de mineração de um bloco usando o algoritmo Proof of Work (PoW) Simples.
4     A mineração envolve encontrar um 'nonce' que, ao ser incluído no bloco, gera um hash que atende a um determinado critério
5     de dificuldade (começando com zeros, por exemplo, dificuldade de 4 o target é "0000").
6     """
7     target = "0" * difficulty
8     # Incrementa o nonce e recalcula o hash do bloco até que o hash resultante, comece igual ao "target".
9     while self.hash[:difficulty] != target:
10         self.nonce += 1
11         self.hash = self._calculate_hash()
12     print(f"Bloco minerado: {self.hash}") # Exibe o hash do bloco que atendeu ao critério de dificuldade (bloco "minerado").
```

Proof of Work (Prova de Trabalho) é o mecanismo utilizado no método `mine_block` para simular o esforço computacional necessário para validar um novo bloco na blockchain.

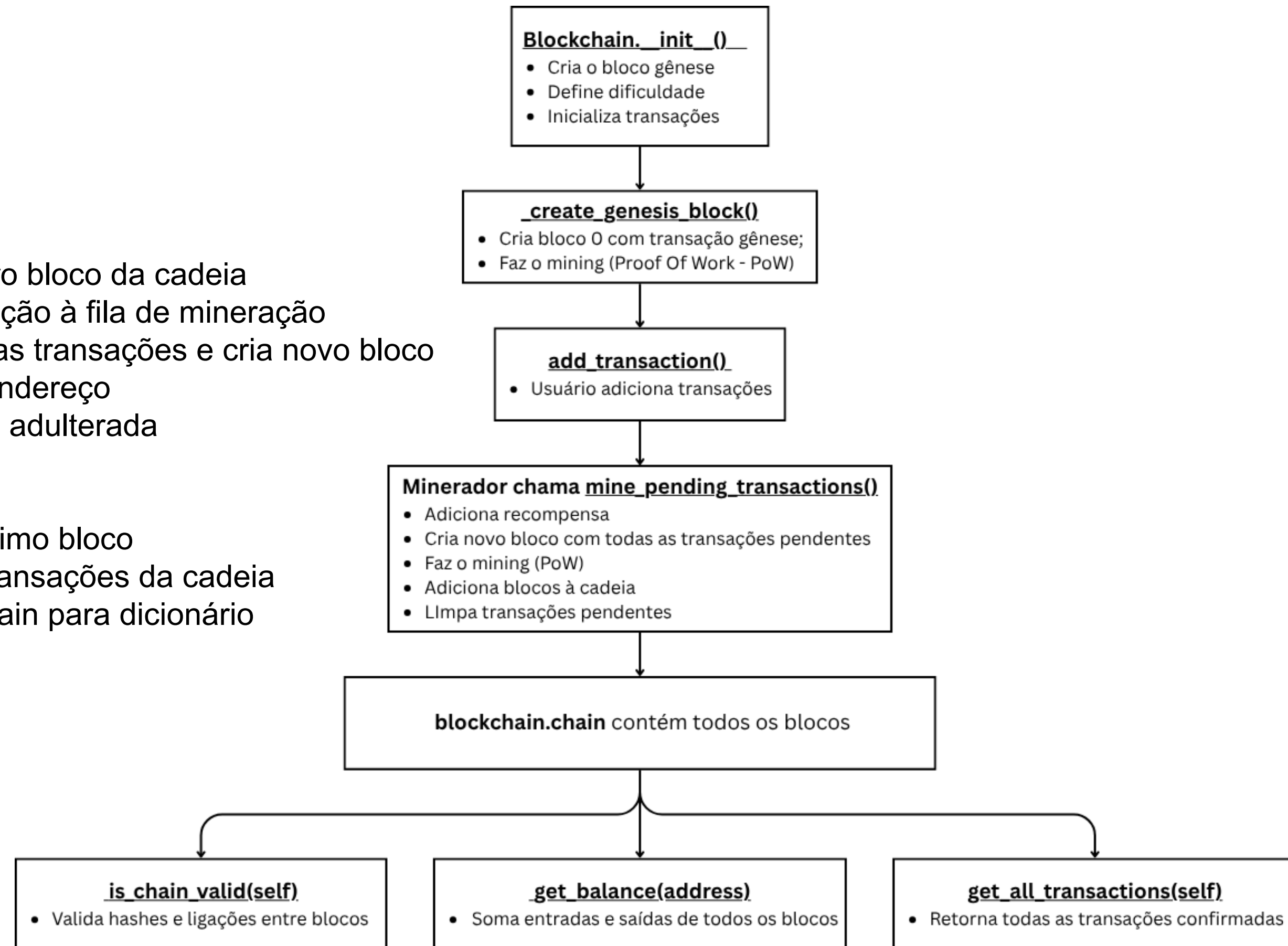
Classe Blockchain

Funções que serão apresentadas

- **__init__** – Inicializa a blockchain
- **_create_genesis_block()** – Cria o primeiro bloco da cadeia
- **add_transaction()** – Adiciona uma transação à fila de mineração
- **mine_pending_transactions()** – Minera as transações e cria novo bloco
- **get_balance()** – Retorna o saldo de um endereço
- **is_chain_valid()** – Verifica se a cadeia foi adulterada

Funções que não serão apresentadas

- **get_latest_block()** – Apenas retorna o último bloco
- **get_all_transactions()** – Lista todas as transações da cadeia
- **to_dict()** – Converte os dados da blockchain para dicionário



Classe Blockchain

```
1 class Blockchain:                                # Implementa um blockchain simplificado.
2     def __init__(self):
3         self.difficulty = 4                        # Define a dificuldade da mineração (Proof of Work)
4         self.pending_transactions: List[Transaction] = [] # Lista de transações pendentes (ainda não mineradas)
5         self.mining_reward = 100                  # Valor da recompensa para quem minera um bloco, ganha 100
6         self.chain: List[Block] = [self._create_genesis_block()] # Cadeia de blocos, iniciada com o bloco gênese
7
8     def _create_genesis_block(self) -> Block:      # Cria o bloco gênese (primeiro bloco).
9         # Cria a primeira transação "genesis", apenas para iniciar a blockchain
10        genesis_transaction = Transaction("genesis", "genesis", 0, "genesis")
11        genesis_block = Block(0, [genesis_transaction], "0") # Cria o bloco gênese com a transação acima e "0" como hash anterior
12        genesis_block.mine_block(self.difficulty)         # Realiza o processo de mineração do bloco gênese
13        return genesis_block
14
15    def mine_pending_transactions(self, mining_reward_address: str)
16        # Cria uma transação de recompensa para o minerador
17        reward_transaction = Transaction(None, mining_reward_address, self.mining_reward)
18        # Adiciona a transação de recompensa às transações pendentes
19        self.pending_transactions.append(reward_transaction)
20        # Cria um novo bloco com as transações pendentes e o hash do último bloco
21        block = Block(len(self.chain), self.pending_transactions, self.get_latest_block().hash)
22        block.mine_block(self.difficulty)              # Realiza a mineração (Proof of Work) do novo bloco
23        self.chain.append(block)                       # Adiciona o bloco minerado à cadeia
24        self.pending_transactions = []                 # Limpa as transações pendentes
25        return block
```

Classe Blockchain

```
1  def add_transaction(self, transaction: Transaction):          # Adiciona uma nova transação à lista de transações pendentes
2      self.pending_transactions.append(transaction)
3
4  def is_chain_valid(self) -> bool:                             # Verifica se a cadeia de blocos está íntegra e não foi adulterada
5      for i in range(1, len(self.chain)):
6          current_block = self.chain[i]
7          previous_block = self.chain[i - 1]
8          if current_block.hash != current_block._calculate_hash(): # Se o hash atual bate com o novo que seria gerado
9              return False
10         if current_block.previous_hash != previous_block.hash:    # Se o hash anterior está correto
11             return False
12     return True
13
14 def get_balance(self, address: str) -> float:                 # Calcula o saldo de um endereço de acordo com o saldo após as transações
15     balance = 0
16     for block in self.chain:                                   # Percorre todos os blocos da cadeia
17         for transaction in block.transactions:                 # Percorre todas as transações de cada bloco
18             if transaction.sender == address:                  # Se o endereço for o remetente, subtrai o valor
19                 balance -= transaction.amount
20             if transaction.receiver == address:                 # Se for o destinatário/receptor, adiciona o valor
21                 balance += transaction.amount
22     return balance
```


Classe BTreeNode

```
1 class BTreeNode:                                # Representa um nó da B-tree.
2     def __init__(self, leaf: bool = False):
3         self.leaf = leaf
4         self.keys: List[Any] = []                # Chaves armazenadas no nó
5         self.values: List[Any] = []              # Valores associados às chaves
6         self.children: List['BTreeNode'] = []    # Lista de Filhos
7
8     def is_full(self, max_keys: int) -> bool:    # Verifica se o nó está cheio.
9         return len(self.keys) >= max_keys        # Se o número de chaves atuais no nó for maior ou igual ao max_keys settado
10
11     def insert_key_value(self, key: Any, value: Any): # Encontra a posição correta para inserir a chave (ordem crescente)
12         index = bisect.bisect_left(self.keys, key) # Retorna qual o índice de uma certa chave
13         if index < len(self.keys) and self.keys[index] == key: # A chave já existe → atualiza o valor
14             if isinstance(self.values[index], list): # Se o valor associado JÁ FOR uma lista (múltiplos valores)
15                 self.values[index].append(value)    # adiciona um novo valor na lista
16             else: # Se o valor associado não for uma lista, for um único valor
17                 self.values[index] = [self.values[index], value] # Converte o valor novo pra uma lista com o valor existente
18         else: # Nova chave → insere na posição correta
19             self.keys.insert(index, key)
20             self.values.insert(index, value)
21
```

Classe BTreeNode

```
--
22 def split(self, max_keys: int) -> Tuple['BTreeNode', Any, Any]:
23     """Divide o nó em dois e retorna o nó direito e a chave/valor do meio."""
24     mid_index = max_keys // 2          # índice do meio, "mediano"
25
26     # Cria um novo nó com as chaves à direita do meio
27     new_node = BTreeNode(leaf=self.leaf)
28     new_node.keys = self.keys[mid_index + 1:]
29     new_node.values = self.values[mid_index + 1:]
30
31     # Se não for folha (se for nó interno), mover também os filhos
32     if not self.leaf:
33         new_node.children = self.children[mid_index + 1:]
34         self.children = self.children[:mid_index + 1]
35
36     # Chave e valor do meio (vão subir para o pai)
37     mid_key = self.keys[mid_index]
38     mid_value = self.values[mid_index]
39
40     # Reduz o nó atual para conter apenas as chaves da esquerda
41     self.keys = self.keys[:mid_index]
42     self.values = self.values[:mid_index]
43
44     return new_node, mid_key, mid_value
```


Classe BTree

Funções principais que serão apresentadas

- **__init__()** - Inicializa a B-tree com raiz folha e define a ordem máxima (quantidade de chaves por nó).
- **insert()** - Insere uma nova chave-valor na árvore. Se a raiz estiver cheia, ela é dividida.
- **_insert_non_full()** - Insere recursivamente em um nó que ainda tem espaço. Evita descer em nós cheios.
- **_split_child()** - Divide um nó cheio e promove a chave do meio para o nó pai, mantendo a B-tree balanceada.
- **search()** / **_search_node()** - Realiza busca eficiente por uma chave, descendo recursivamente até encontrá-la ou concluir que não existe.

Funções que não serão apresentadas:

- **range_search()** - Realiza busca em intervalo.
- **get_all_items()** - Apenas coleta todos os dados da árvore em ordem crescente, utilizando travessia.
- **_inorder_traversal()** - Apoia get_all_items(). Percorre os nós da árvore em ordem (filho esquerdo → chave → filho direito).
- **print_tree()** / **_print_node()** - Usadas para depuração e visualização da estrutura.

Classe BTree

```
1 class BTree:                                # Representa a B-tree.
2     def __init__(self, max_keys: int = 5):    # Por default é de ordem = 5
3         self.root = BTreeNode(leaf = True)   # Cria a raiz como um nó folha
4         self.max_keys = max_keys              # Número máximo de chaves por nó (ordem da árvore)
5         self.min_keys = max_keys // 2        # Número mínimo de chaves (usado em splits e balanceamento)
6
7     def _split_child(self, parent: BTreeNode, child_index: int):    # Divide um nó filho que está cheio
8         child = parent.children[child_index]
9         # Divide o nó e obtém o novo nó e chave do meio
10        new_child, mid_key, mid_value = child.split(self.max_keys)
11        # Insere a chave do meio no pai
12        parent.keys.insert(child_index, mid_key)
13        parent.values.insert(child_index, mid_value)
14        # Insere o novo filho à direita do filho original
15        parent.children.insert(child_index + 1, new_child)
16
17    # Funções de busca
18    def _search_node(self, node: BTreeNode, key: Any) -> Optional[Any]: # Busca em um nó específico na BTree
19        i = 0
20        while i < len(node.keys) and key > node.keys[i]:
21            i += 1
22        if i < len(node.keys) and key == node.keys[i]:
23            return node.values[i]
24        if node.leaf:
25            return None
26        return self._search_node(node.children[i], key)
27
28    def search(self, key: Any) -> Optional[Any]:    # Busca uma chave na árvore (pública)
29        return self._search_node(self.root, key)
```

Classe BTree

```
30
31 def insert(self, key: Any, value: Any):      # Insere uma nova chave-valor na árvore
32     root = self.root
33     if root.is_full(self.max_keys):          # Se a raiz está cheia, precisamos dividir
34         new_root = BTreeNode(leaf=False)    # Cria uma nova raiz
35         new_root.children.append(self.root)  # Eleva a raiz antiga a um filho
36         self._split_child(new_root, 0)       # Divide a raiz antiga
37         self.root = new_root                 # Atualiza a raiz
38     self._insert_non_full(self.root, key, value) # Chama o método auxiliar pra inserir chave-valor no nó que não está cheio
39
40 def _insert_non_full(self, node: BTreeNode, key: Any, value: Any):    # Inserção em um nó que ainda tem espaço
41     if node.leaf:              # Inserir diretamente, pois é uma folha
42         node.insert_key_value(key, value)
43     else:                      # Achar o filho correto, pois é um nó interno
44         child_index = 0
45         while child_index < len(node.keys) and key > node.keys[child_index]:
46             child_index += 1
47         child = node.children[child_index]
48         if child.is_full(self.max_keys):    # Se o filho estiver cheio, divide primeiro
49             self._split_child(node, child_index)
50             if key > node.keys[child_index]: # Verifica novamente para qual filho ir
51                 child_index += 1
52         child = node.children[child_index]
53     self._insert_non_full(child, key, value)
54     # Continua a inserção recursivamente, até alcançar uma folha não cheia e inserir a chave
```